Merkle Commitments for Arkworks

Alexander J. Havlin
alexander.havlin@epfl.ch
EPFL

Andrew Zitek-Estrada
andrew.zitek@epfl.ch
EPFL

June 2026

Abstract

Merkle commitment schemes are increasingly essential tools in the construction of post quantum secure cryptography in general and zero knowledge proof systems specifically. While widely accepted techniques are intuitive to describe, their apparent simplicity disguises the potential for critical parameterization errors. Practitioners are pulled in two directions: oversized parameters make the scheme slow, while under configuration leads to concrete vulnerabilities in the higher level protocol. In this effort, we address these challenges leveraging an interface that bridges security definitions with scheme configuration. We implement this infrastructure in Rust for the Arkworks ecosystem and show that undersized concrete instantiations (based on a security parameter) are detected and refused rather than shipped. Additionally, we provide methodology for describing the relationship between security and performance for properties like binding and hiding. Finally, we provide machine-checked analysis of the binding property as described in this framework using Lean 4.

1 Contributions

AZ - I don't think this paragraph is doing work for you. I'd probably scrap entirely revise how you feel appropriate the new version I'm suggesting for you below.

Merkle commitments are the hash-based vector commitments that post-quantum proving systems are compiled through, so the commitment's guarantees become the compiled argument's guarantees. Configuring one is subtle: the digest width and the per-leaf salt cardinality both move with the field, and an undersized choice still compiles and passes every functional test. The defaults shipped by Arkworks miss this detail, which comes to matter when working in small fields. This project makes sizing those parameters the infrastructure's obligation rather than the user's. The contributions:

AZ - Merkle commitments are essential to modern proof systems, but practioners have an unreasonable challenge of balancing security and performance in the context of complex security definitions and extensive configuration levers. We aim to reduce this burden on the user by lifting it into infrastructure. With this mind, we highlight four main contributions:

AZ - I find all of these titles hard to understand.

- **A parametrisation function for the commitment layer** (Sections 2 and 5): for a chosen field, hash, interface, and adversary query budget, enter a target security level and receive the digest width and salt cardinalities that achieve it, or enter concrete parameters and receive the security level they achieve together with an estimate of their cost. Computed once, reused across fields, independent of tree arity.
  AZ - What "shipped defaults"? That repo isn't published. I would probably remove this.
- **A demonstrated break of the shipped defaults**: the field-size-agnostic one-element digest publishes a BabyBear root in a ∼30-bit space, and a deliberate collision (one root, two accepted openings disagreeing at a shared index) takes 44 seconds (Section 3); the matching one-element salt breaks hiding outright (Section 4).
  AZ - This is probably good
- **A measured cost matrix** (Section 6): we benchmark prover time, verifier time, and proof size across parameter combinations spanning three fields and two hash families, separating security thresholds from

cost transitions and showing that for Poseidon2 the certified $\lambda = 128$ parameters cost nothing extra (Table 1).
AZ - You can keep this but I don't really know what you mean by "appropriate ROM" maybe just pick one and keep it simple?

- **A machine-checked proof line**: we are developing a Lean 4 proof [z-l26] that the derived parameters give binding and hiding in the appropriate random-oracle model; the parts not yet mechanised are either established by other means or rest on theorems already known and accepted in the literature.
  AZ - consolidation of optimal multiproof impl <– you can keep this but make it very brief
- **A consolidation of the first semester's engineering**: the multiproof compression and its coordinate-encoding backing (out of scope here, but the groundwork for geometric binding), the interface redesign, and the benchmark harness built against the legacy Arkworks/crypto-primitives (Section 7) are carried into this report, which supplies the specification that work was missing.

AZ - Delete. To draw the line of contribution plainly: the rule, the break, the benchmark matrix and its harness lineage (Section 7), and the formalisation are this work. The vector-commitment and Merkle notation and the concrete-security accounting are [CY26]'s, and the ark-vc crate design belongs to the surrounding project.

2 Background

AZ - What two properties? I really don't understand this section it's a firehose of out of context facts and sentences

What the two properties buy. In the BCS compilation [BSCS16] ark-vc [Ark26] targets, in the lineage of Kilian's committed-proof arguments and Micali's CS proofs [Kil92, Mic00, FS87], each oracle message is committed and the verifier's queries are answered with authenticated openings. The root is fixed into the transcript from the first message, so *binding* enters the compiled soundness bound additively [CDGS24] and lets the soundness argument treat committed answers as fixed, while Fiat–Shamir grinding inflates the query budget and raises the digest target from roughly 2λ to 3λ bits. *Hiding* is what a zero-knowledge argument relies on, and it has no protocol-level rescue: the root is read before any later message, so salt entropy cannot be traded for code distance, query count, or round structure.

Why a rule is needed. The shipped failure is concrete. At the `MerkleHasher` trait of ark-mt the digest and salt widths are associated types with no field-size check, and for BabyBear at $\lambda = 128$ a one-element digest publishes a root in a $\sim$30-bit space. An honest tree over 2^{20} leaves already makes roughly 2 million internal hash calls, far past that space's $\sim$46,000 birthday threshold, and a deliberate collision confirms the binding break in 44 seconds; the one-element salt fails hiding the same way. Neither failure implicates the hash function. Both follow from parameters that do not scale with the field, which is exactly what a derivation rule would prevent and why it is important that users be made aware.

The design rule. A protocol designer supplies five things: a field $\mathbb{F}$, a security level λ, a hash H, a choice of standard or hiding interface, and an adversary model, declared as the query-budget exponent h. Everything else is derived. From $(\mathbb{F}, \lambda, h)$ the parameter constructor produces

$$D = \left\lceil \frac{\lambda + 1 + 2h}{b} \right\rceil, \qquad k = \left\lceil \frac{\lambda}{b} \right\rceil, \qquad S = \left\lceil \frac{\lambda}{8} \right\rceil, \tag{1}$$

where $b = \lfloor \log_2 |\mathbb{F}| \rfloor$ is the conservative bits-per-element. Here D is the digest width in field elements, and k, S are the field- and byte-salt cardinalities. These are not recommendations: each is a checkable obligation that the constructor either meets, emitting the derived parameters, or refuses, so a designer can neither quietly under-specify nor silently over-claim.

The justification is short. A $q \le 2^h$-query collision search succeeds against a κ-bit digest with probability

about $q^2/2^{\kappa+1}$, so meeting $2^{-\lambda}$ needs $\kappa \geq \lambda + 1 + 2h$ bits, and D is that floor in field elements; k and S give the salt at least λ bits of entropy in the hash's native unit. The query-budget exponent h is the one input that names the adversary model, which is why it is declared rather than derived (Section 3 prices two choices of it). One scope note applies throughout: every parameter is sized against a classical adversary, since Grover search halves effective salt entropy and quantum collision search shifts the digest accounting, and a quantum column of the rule is deferred. The report follows the rule by property: binding (Section 3) and hiding (Section 4) carry their own definition, failure, and parameter, Section 5 states the model behind the rule, and Section 6 measures the cost.

AZ - If this is truly meant to be background ... you want to explain that traditionally people think about merkle commitments as a tree with arity 2 and a single hasher type f at every level. More recently, people have an interest in changing arity and hashers at different levels for performance and recursion friendliness and this adds complexity to security analysis.

Then you could very briefly summarize basically the book's coverage of whatever properties are most important: binding, hiding, whatever you think is sufficient and in scope.

Key definitions can go here but they need to be much much more concise then they currently read.

Also, if you really want to talk about BCS it needs to be incredibly succinct and I don't see how you can do this without before defining an IOP as a popular construction of zero knowledge proof systems in like two sentences or less. I leave this to you to decide but I have my doubts.

Then you're done. This is the background of what a merkle commitment is and what about it is important.

AZ - Next, you want to walk through each of your four contributions as one section each (in whatever order you find best).

3 Binding

3.1 Definition and Failure

Definition 3.1 (Position binding)**.** VC is *position binding* if no efficient adversary outputs a commitment cm with two accepted openings $(I_b, \mathbf{a}_b, \mathsf{pf}_b)$ that disagree at some $i \in I_0 \cap I_1$, except with probability $\mathsf{negl}(\lambda)$. The commitment has no honest-generation requirement. A single conflict at a shared index is the local witness that binding has failed.

One conflict is the whole failure. Position binding is what lets a soundness argument treat each queried answer as the unique value the commitment fixes the moment cm is published, before the query set is chosen.

3.2 Sizing and Validation

Two adversary models, two digest targets. The concrete-security accounting behind the digest floor is the worst-case-versus-average-case split of [CY26]: at $\lambda = 128$, binding against a worst-case adversary needs a 383-bit output, while the average-case notion needs 255 bits, a roughly 1.5× gap. The grinding discussion of Section 2 is that worst-case notion stated operationally, because a grinding prover is exactly one who spends its whole query budget hunting for the transcript where the bound is tightest.

The rule prices both, through h. The rule (Equation (1)) sets the digest to $D = 9$ field elements ($\kappa = 270$ bits) for BabyBear at $\lambda = 128$, $h = 64$: a 13-bit margin over the $\lambda + 1 + 2h = 257$-bit floor, which is the average-case instance. The worst-case target enters the rule only through the query budget h, the declared input that names the adversary model. Re-running the constructor at $h = 128$ gives $D = \lceil (128 + 1 + 256)/30 \rceil = 13$, so $\kappa = 390 \geq 383$, with no new analysis. $D = 9$ and $D = 13$ are two certificates for two adversary models and must not be crossed: a $D = 9$ configuration certifies nothing against a grinding prover. Write Θ for the concrete configuration under test (field, level, hash, and derived widths), so $\mathsf{Adv}^{bind}_{\mathsf{VC},\mathcal{A}}(\Theta)$ is the binding advantage against that Merkle instance.

Proposition 3.2 (Binding from collision resistance)**.** *Let $\mathsf{MT}[H]$ be a Merkle commitment over a shape G_ℓ whose leaf encoding is injective and whose leaf and internal-node calls are domain separated. If the shared*

κ-bit oracle receives at most q distinct queries during the complete experiment, then

$$\mathsf{Adv}^{bind}_{\mathsf{VC},\mathcal{A}}(\Theta) \le \Pr[\textit{oracle-trace collision}] \le \frac{q(q-1)}{2^{\kappa+1}},$$

which meets $2^{-\lambda}$ when $q \le 2^h$ and $\lambda + 1 + 2h \le \kappa$. [Lean-backed, generic over `MerkleShape`]

Proof sketch. A disputed index $i \in I_0 \cap I_1$ yields two authentication paths that agree at some parent but disagree below it: two distinct oracle inputs with one output. Domain separation pins the leaf and internal roles, so the collision is genuine. What remains is the adversary's freedom to re-interpret the same digests under a different tree shape, which is the tree-inequality event E_{tree} of [CY26]: the tree reconstructed from the commitment trace and the one reconstructed after the opening queries can differ only if some opening answer accidentally matches a label already fixed in the first tree. A union bound over the opening queries keeps that probability negligible whenever the digest resists collisions, so a binding break still reduces to one oracle collision rather than to any freedom in how the tree is read. Hashing a geometry tag of layer, index, and arity into each node keeps this domain separation positional, which forecloses the replay of a subtree proof in a differently shaped tree.

The reduction is generic; the backing is mechanical. The colliding-paths step, the birthday bound, and the parameter arithmetic additionally carry a machine-checked Lean backing [z-l26], so binding is the axis with independent mechanical assurance behind it. The reduction itself is generic over the Merkle shape: the binary tree is the exhibited instance, and k-ary and arbitrary-length shapes share the argument but are not yet exhibited (Section 8).

4 Hiding

4.1 Definition and Failure

Definition 4.1 (Hiding)**.** A randomised VC whose Commit draws salts from a space $\mathcal{R}$ is *hiding* if, for any two messages of length ℓ and any adversary that after seeing cm adaptively opens positions on which the messages agree, the advantage in deciding which was committed is at most $\mathsf{negl}(\lambda)$. For Merkle this refines to *root hiding*—the real root is statistically close to one simulated without $\mathbf{m}$—strengthened to *selective-opening hiding*, which must also hide through the disclosed salts and authentication paths of partial openings.

A Merkle salt is a random per-leaf nonce: each leaf is itself a basic commitment $H(\mathbf{m}_i, \text{salt})$, and the salt travels in the authentication path beside the sibling digests so the verifier can recompute the leaf. Opening one leaf commitment under two different salts would force a hash collision, so the salt binds a value to its position even as it hides it. Hiding then rests entirely on the salt space, which must be both well sampled and large enough to resist search within the security budget. A deterministic salt satisfies neither, since the receiver can recompute candidate leaf commitments. Unlike binding's additive contribution, there is no protocol-level rescue, because the root is read from the transcript before any later message.

4.2 Sizing and Contract

The rule (Equation (1)) sets the salt cardinality to $k = 5$ field salts or $S = 16$ byte salts for BabyBear at $\lambda = 128$, each defining a space of at least 2^{128} elements. These cardinalities are machine-checked.

What remains is an *intended reduction*, not a machine-checked theorem. The root-hiding obligation has the form

$$\mathsf{Adv}^{hide}_{\mathsf{VC},\mathcal{A}}(\Theta) \le \frac{\ell q}{2^s} + \varepsilon_{\text{internal}}(\kappa, q, \mathsf{shape}),$$

where the terms that appear represent the two bad events: a query to a hidden salted leaf input, or an internal hash collision as the simulation rebuilds the tree from the leaves up to the root. The bound is written against the intended game, in which the honest experiment samples the oracle jointly with the salts, and discharging that game is the open step. The salt-hit term rests on a standard averaging step that is not yet formalised. The certified salt cardinalities are prerequisites for this bound, not the theorem, and lifting the result to the selective-opening target is the composition step after that.

5 The Parametrisation Model

Binding and hiding failed separately and were sized separately, but the two sizings are one rule, and that they share a rule is the point of this section. The earlier Arkworks/crypto-primitives [Ark25] bought security by hardcoding every choice, and developers, wanting simpler hash interfaces, variable tree shapes, and efficient multi-opening compression, asked for that freedom back. Granting it lets a user control $(\mathbb{F}, \lambda, H, \text{interface})$ in full, and the danger is exactly that without hardcoded guardrails a misconfiguration stays silent. So the developer faces a question they cannot answer from the interface alone: which security claim does my concrete configuration actually support? Answering it is our obligation, not theirs. Given those choices and a declared adversary model, the infrastructure owes back the parameters that must be used, a proof that they meet the target, the cost they incur, and the property they buy. The user can then keep their attention on their own implementation and read its concrete security off the guarantees we give.

5.1 The Object It Sizes

We fix the commitment object once, following the Chiesa–Yogev notation so the mathematical boundary does not drift from the reference the implementation follows [CY26].

Definition 5.1 (Vector commitment scheme)**.** A *vector commitment scheme* [CF13] over alphabet Σ and length ℓ is a triple $\mathsf{VC} = (\mathsf{Commit}, \mathsf{Open}, \mathsf{Check})$:

- $\mathsf{Commit}(\mathbf{m}) \to (\mathsf{cm}, \mathsf{aux})$ binds a message $\mathbf{m} \in \Sigma^\ell$ to a short commitment cm and retains committer state aux;
- $\mathsf{Open}(\mathsf{aux}, I) \to \mathsf{pf}$ authenticates an index set $I \subseteq [\ell]$;
- $\mathsf{Check}(\mathsf{cm}, I, \mathbf{a}, \mathsf{pf}) \to \{0, 1\}$ decides whether pf authenticates the subvector $\mathbf{a}$ at I against cm, where for an honest committer $\mathbf{a}$ is exactly the committed message $\mathbf{m}$ restricted to the positions in I.

A scheme is *complete* (Definition 5.2) and *position-binding* (Definition 3.1), and may implement optional extensions—hiding, treated here, plus updatability, aggregation, functional opening, and knowledge soundness—which ark-vc's capability model accommodates but this report does not treat.

Definition 5.2 (Completeness)**.** VC is *complete* if for every $\mathbf{m} \in \Sigma^\ell$ and every $I \subseteq [\ell]$ the honest Commit–Open–Check round trip accepts $\mathbf{m}_I$ with probability 1. For Merkle this is honest reconstruction of the root from an opening.

Definition 5.3 (Merkle commitment scheme)**.** The *Merkle commitment scheme* [Mer88] $\mathsf{MT} = (\mathsf{MT.Commit}, \mathsf{MT.Open}, \mathsf{MT.Check})$ instantiates Definition 5.1 in the random-oracle model over a general Merkle shape G_ℓ of depth $d = \max_i d_i$, the deepest of the per-leaf depths d_i over leaves i: the three algorithms query a shared oracle f, the commitment is the tree root, and the committer state is the retained salts and labels. The binary tree is the arity-2 subset of this shape. Two parameters carry the security: the digest width κ governs binding and the salt size s governs hiding, with no tuning of one supplying what the other lacks.

Limitation 5.4 (No trapdoor, so no standard-model equivocation)**.** MT has no cryptographic Setup: the public parameter is the hash itself, so the trapdoor is empty ($\mathsf{td} = \perp$). This shapes what can even be asked of the scheme. Extraction cannot program a trapdoor and must instead argue from the adversary's random-oracle query trace, and standard-model equivocation, which opens one commitment to two values precisely by programming a trapdoor, has nothing to program. Equivocation is therefore unreachable for this scheme by construction, not merely unproved.

5.2 The Rule

The rule itself was stated in the box of Section 2: from the designer's interface-level inputs $(\mathbb{F}, \lambda, H, \text{interface})$ and the declared adversary budget h, the constructor derives the digest width D, the field-salt cardinality k, and the byte-salt cardinality S of Equation (1), and either meets each obligation or refuses the configuration. What remains to establish here is what the rule does not depend on, and what it produces per field.

The rule is arity-independent. (D, k, S) depends only on $(\mathbb{F}, \lambda, h)$, while the tree shape and arity enter only through the query budget h and the cost. The same target therefore sizes any Merkle shape and produces

Property	Field	Parameter	Prover (ms)	Verifier (ms)	Proof (kB)
Hiding	BabyBear	$k = 5$	25.7	1.03	3.6
Hiding	Goldilocks	$k = 3$	39.8	1.51	4.6
Hiding	BN254	$k = 1$	175.2	6.56	10.3
Binding	BabyBear	$D = 9$	38.7	1.84	10.1
Binding	Goldilocks	$D = 5$	40.3	1.37	11.0
Binding	BN254	$D = 2$	260.0	11.26	16.4

Table 1: Measured cost of the first Poseidon2 configuration reaching $\lambda \geq 128$ per field, for the hiding benchmarks (varying the salt cardinality k) and the binding benchmarks (varying the digest width D). Prover time is mean commit plus open; verifier time is mean check; proof size is the compressed proof. Fixed regime as in Figure 1: binary tree, 4,096 leaves, 32 openings.

different outputs per field: for BabyBear ($b = 30$, $h = 64$) it gives $D = 9$, $k = 5$, $S = 16$; for BN254 ($b \approx 254$) it gives $D = 2$, $k = 1$, $S = 16$. These are not interchangeable, and the 44-second break of Section 2 used a $D = 1$ configuration, one of the hashers Arkworks currently ships, which is exactly the kind of undersizing the constructor now refuses.

6 Cost

What the benchmarks measure. The rule says which configurations support the claim; the benchmarks say what they cost. Each plot fixes a binary tree with 4,096 leaves and 32 openings and reports mean prover time, verifier time, and proof size. A field-coloured line marks the first $\lambda \geq 128$ configuration, and a red × marks each cost transition.

The salt arrives in the hash's native unit. The hash takes its salt in its own native unit: k field elements for an algebraic hash such as Poseidon2 [GKS23], or S bytes for a byte-oriented hash such as Blake3 [OANWO20]. Because the unit already matches the hash, the salt pays no encoding penalty. The rule therefore reports two salt columns, k and S, not as redundant choices but as the right one for each kind of hash.

Where the thresholds land. The Poseidon2 hiding benchmarks reach $\lambda = 128$ at $k = 5$ for BabyBear, $k = 3$ for Goldilocks, and $k = 1$ for BN254; the binding benchmarks at $D = 9$, $D = 5$, and $D = 2$. Table 1 reports the absolute numbers at those crossings, so the panels of Figure 1 can be read for shape rather than for values.

For Poseidon2, certified security is free. For Poseidon2 both the $\lambda \geq 128$ and $\lambda \geq 256$ thresholds land before the first red × for all three fields. The certified parameters fit inside the permutation block already computed for the rate-width output, so correct security costs nothing extra: widening the digest up to the permutation rate reuses output that is computed anyway (Figure 2, left). Bytewise Blake3 is the contrast: only the 128-bit hiding salt clears before its first compression-block transition, while wider digests cross a block boundary and pay with an extra Blake3 compression block (Figure 2, right).[1] [empirical; binary tree, 4,096 leaves, 32 openings]

7 Provenance: From Retrofit to Parametrisation Model

The first phase of this work compared the existing Arkworks/crypto-primitives Merkle tree with the Chiesa–Yogev presentation [CY26]. It began as implementation work around open feature requests: simplify the split hash interface, remove caller-side ordering footguns, support richer tree shapes, and compress multi-openings.

[1] The full bytewise Blake3 benchmark results are plotted in the same triptych format as Figure 1 and ship with the report sources; the prose here summarises their result.

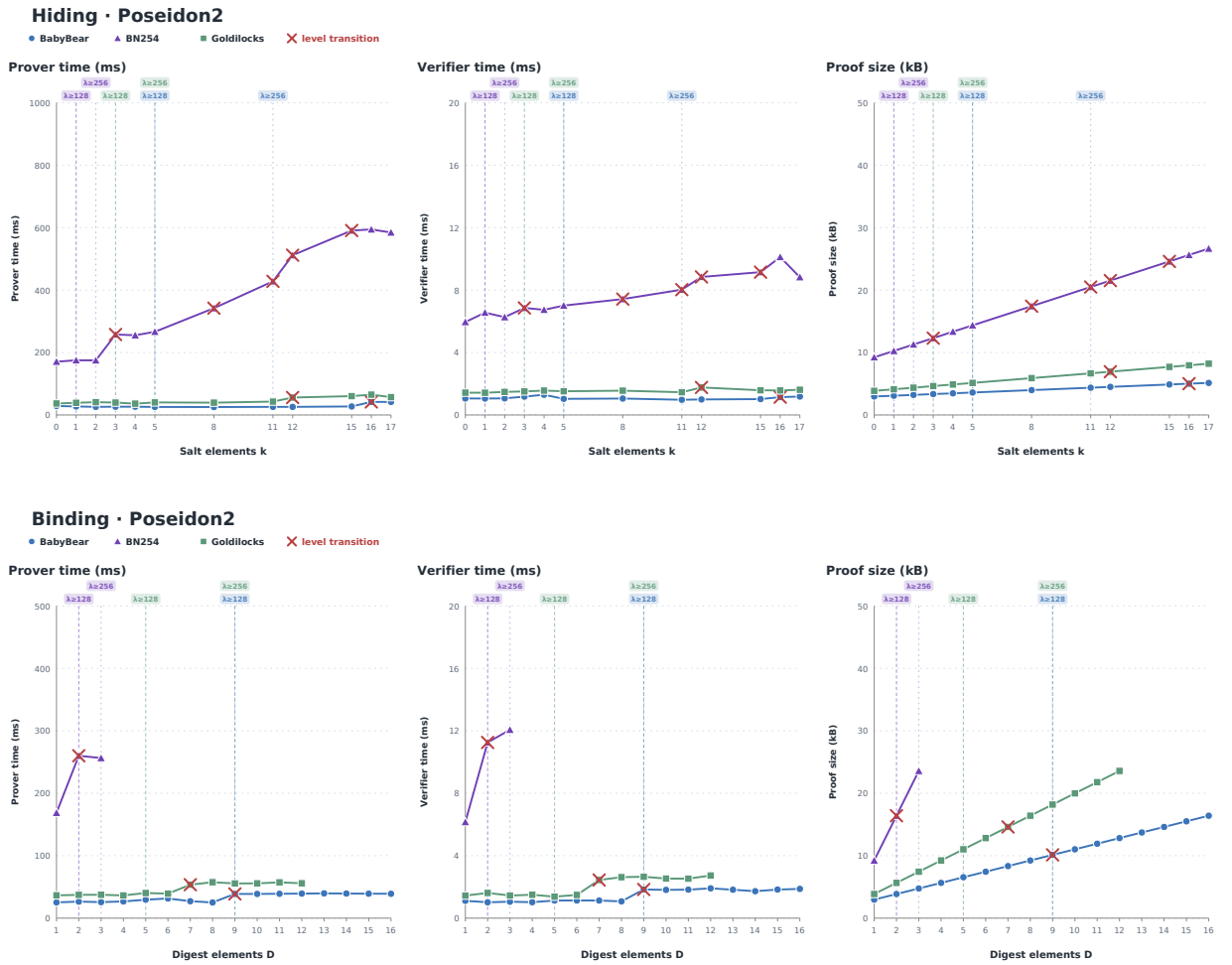

Figure 1: Poseidon2 hiding cost as salt width grows (top) and binding cost as digest width grows (bottom). Field-coloured lines mark the first measured $\lambda \geq 128$ and $\lambda \geq 256$ points for BabyBear, Goldilocks, and BN254; red x markers flag inputs that cross into an extra permutation.

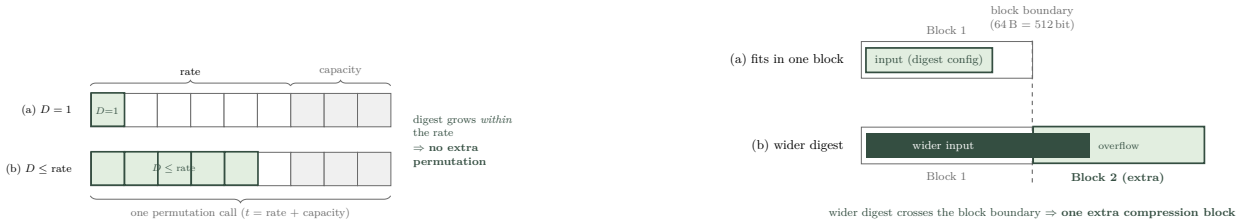

Figure 2: Why the cost transitions fall where they do. Left: a Poseidon2 permutation writes its output into the rate of a fixed-width state, so widening the digest up to the rate stays inside one permutation call and is free. Right: Blake3 absorbs in fixed blocks, so a digest that crosses a block boundary costs one extra block.

Read together, those requests pointed to one broader issue. The implementation had many locally reasonable decisions, but no single specification against which their combined behaviour could be reviewed.

That comparison produced concrete engineering results. The `CoSet` multiproof encoding realises the book's path-pruning optimisation for multi-openings, and `ImplicitCoPath` performs the same pruning without materialising a coordinate bookkeeping layer, which is purely pedagogical when used outside a position-tagging context. That coordinate-encoding backing was nonetheless built out in full, and although it sits outside this report's use, it is exactly the groundwork the geometric, position-tagged style of binding (Section 3) would

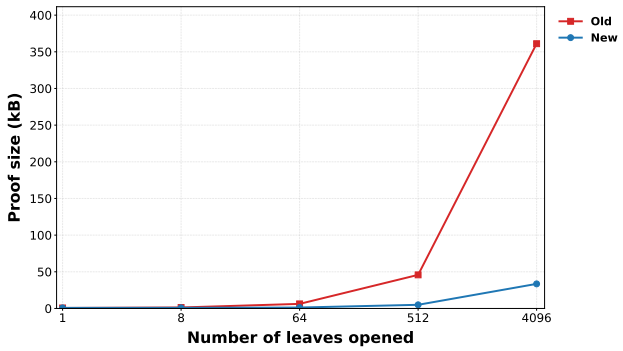

Figure 3: Phase-1 multiproof compression on clustered openings of a 4,096-leaf tree. The `CoSet` encoding prunes shared path material, so proof size grows with the disclosed frontier rather than with the number of individual paths: at 4,096 opened leaves the compressed proof is roughly 33 kB against roughly 360 kB uncompressed.

build on. The `Committed/OpeningProof/Opening` split makes ordering and duplicate-index validation part of construction rather than caller discipline. Figure 3 shows the headline measurement: at 4,096 opened leaves the clustered multiproof is roughly a tenth the size of the per-path encoding it replaces. The benchmark harnesses built to take that measurement prefigure the cost benchmarks of Section 6. None of this code was merged into the current tree.

The stronger finding was methodological. A reference definition can organise an API, yet an API alone still does not tell a developer which security claim a concrete configuration supports. That is the step from retrofit to parametrisation model: keep the literature-facing definition, connect it to executable interfaces, state the security games, derive the parameters, and expose the evidence boundary. What the semester enabled therefore runs through the whole report. The freedoms whose absence the retrofit exposed (simpler hash interfaces, variable tree shapes, efficient multi-opening compression) are exactly the ones the parametrisation model of Section 5 grants and then guards. The multiproof pruning strategy survives in ark-mt's capped openings, where the commitment publishes the ordered digest layer at a chosen depth instead of the single root, so every opening sheds that many layers of shared copath. The cost measurements of Section 6 run on the benchmarking craft built here.

8 Conclusion and Next Steps

A vector commitment is only as strong as the parameters underneath it, and the default ark-mt configurations miss that target for small fields on both axes at once. This report established three things. The rule of Equation (1) derives, from a designer's declared inputs, the digest and salt parameters a security target forces, and refuses the undersized configurations the shipped defaults accept, including the one broken here in 44 seconds. The cost benchmarks price the derived parameters and show that for Poseidon2 the certified configurations fit inside permutation blocks already paid for, so correct security at $\lambda = 128$ costs nothing extra. The binding analysis behind the rule additionally carries a machine-checked Lean backing.

Next. The formal status splits three ways, and the split is the information. Closed: the binding chain end to end, and the salt-cardinality lemmas hiding needs. Scoped and believed but not yet closed: the extraction bridge and one distributional averaging step, both reduced to named deterministic pieces. Not yet stated in the formal model: the sampling-based hiding game and equivocation, so the hiding claim rests on the intended reduction of Section 4 rather than a checked theorem. Beyond the formalisation, multi-round BCS and Fiat–Shamir soundness would justify the strengthened digest target at the protocol level, and the k-ary and arbitrary-length shapes share the generic binding reduction but are not yet exhibited.

References

[Ark25] Arkworks contributors. Arkworks/crypto-primitives: Cryptographic primitives for the arkworks ecosystem. https://github.com/arkworks-rs/crypto-primitives/, 2025.

[Ark26] Arkworks contributors. ark-vc: Vector commitment interfaces and Merkle instantiations. https://github.com/arkworks-rs/z/, 2026. Crate under development in the arkworks project.

[BSCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. Interactive oracle proofs. In *Theory of Cryptography (TCC)*, volume 9986 of *LNCS*, pages 31–60, 2016.

[CDGS24] Alessandro Chiesa, Marcel Dall'Agnol, Ziyi Guan, and Nicholas Spooner. On the security of succinct interactive arguments from vector commitments, September 2024. Manuscript.

[CF13] Dario Catalano and Dario Fiore. Vector commitments and their applications. In *Public-Key Cryptography (PKC)*, volume 7778 of *LNCS*, pages 55–72, 2013.

[CY26] Alessandro Chiesa and Eylon Yogev. *Building Cryptographic Proofs from Hash Functions*. 2026. Draft.

[FS87] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *CRYPTO '86*, volume 263 of *LNCS*, pages 186–194, 1987.

[GKS23] Lorenzo Grassi, Dmitry Khovratovich, and Markus Schofnegger. Poseidon2: A faster version of the Poseidon hash function. In *AFRICACRYPT 2023*, volume 14064 of *LNCS*, pages 177–203, 2023.

[Kil92] Joe Kilian. A note on efficient zero-knowledge proofs and arguments. In *Symposium on Theory of Computing (STOC)*, pages 723–732, 1992.

[Mer88] Ralph C. Merkle. A digital signature based on a conventional encryption function. In *CRYPTO '87*, volume 293 of *LNCS*, pages 369–378. 1988.

[Mic00] Silvio Micali. Computationally sound proofs. *SIAM Journal on Computing*, 30(4):1253–1298, 2000.

[OANWO20] Jack O'Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O'Hearn. BLAKE3: One function, fast everywhere, 2020.

[z-l26] z-lean contributors. VectorCommitment: Executable Merkle commitments and security proofs in Lean. https://github.com/z-tech/z-lean/, 2026.